

FINE-TUNER UN MODÈLE DE LANGAGE

De l'architecture Transformer au déploiement d'un modèle adapté à un cas métier

FORMAT

7h présentiel · 4h distanciel · 45min tutorat

MODÈLE

Llama 3.2 1B + LoRA

COMPÉTENCES

C5 — Entraîner · C6 — Implémenter et exposer

CAS D'USAGE

Triage de demandes clients FastIA

Le mécanisme d'attention

"La banque a refusé le prêt car **elle** manquait de fonds."

Pour comprendre "**elle**", le modèle pose une requête à tous les autres tokens et calcule un score de pertinence pour chacun.

Token	Score d'attention
banque	0.82 — fort
refusé	0.09
prêt	0.06
fonds	0.03

Score élevé = le contenu (V) de ce token est fortement transmis pour construire la représentation de "elle".

Q — "elle" pose une question
vecteur appris : "à qui est-ce que je réfère ?"

comparé à chaque K de la séquence

K — vecteur appris par chaque token
"banque" a appris à signaler : je suis un sujet potentiel

$Q \cdot K$ élevé $\rightarrow$ score fort $\rightarrow$ V transmis

Sortie — "elle" = une banque
représentation enrichie du contexte

Q, K et V sont des vecteurs calculés via des matrices de projection apprises pendant le pré-entraînement.

Un triplet Q/K/V = **une tête d'attention**. Llama 3.2 1B en exécute 32 en parallèle — chacune apprend à "regarder" un aspect différent du contexte.

Architecture d'un Transformer decoder

Texte en entrée → tokens



Embedding + Positional Encoding



Bloc Transformer × N
Self-Attention + Feed-Forward



Token suivant prédit

EMBEDDING

Chaque token est converti en vecteur. Des tokens proches en sens ont des vecteurs proches dans l'espace.

POSITIONAL ENCODING

Comme l'attention voit tout en simultané, on ajoute l'information de position pour que l'ordre des mots compte.

LLAMA 3.2 1B

16 blocs Transformer empilés. Chaque bloc affine la compréhension du contexte. 1 milliard de paramètres au total.

Encoder vs Decoder — deux familles

ENCODER — BERT, CAMEMBERT, ROBERTA

Lit la séquence entière dans les deux sens

Produit une représentation du texte

Tâches : classification, NER, similarité sémantique

Ne génère pas de texte

DECODER — LLAMA, GPT, MISTRAL, QWEN

Prédit le token suivant, séquentiellement

Génère du texte token par token

Tâches : génération, instruction following, classification via prompt

Architecture dominante aujourd'hui

Dans ce module, on travaille avec un Decoder : **Llama 3.2 1B**. On lui apprend à générer une réponse JSON structurée à partir d'une instruction métier.

Ce que Llama 3.2 1B sait déjà

Données internet

Wikipedia, livres, code, web — centaines de milliards de tokens

↓ semaines de calcul — centaines de GPU

Pré-entraînement par Meta



Modèle de base Llama 3.2 1B

Disponible sur HuggingFace Hub

CE QU'IL A APPRIS

Grammaire, syntaxe, faits du monde, raisonnement, structure du langage, code, mathématiques...

CE QU'IL NE SAIT PAS ENCORE

Votre domaine métier spécifique, votre format de sortie attendu, vos conventions et règles propres à FastIA.

Les trois approches pour adapter un modèle

PROMPT ENGINEERING

Aucun entraînement

On guide le modèle avec des instructions précises

Limites : peu fiable, sorties imprévisibles, dépend du modèle

Module 0

FINE-TUNING

Réentraînement sur des données métier

Le modèle apprend un comportement précis et reproductible

Sorties maîtrisées, format garanti

Module 1 — aujourd'hui

ENTRAÎNEMENT FROM SCRATCH

Partir de zéro

Nécessite des milliards de tokens et des semaines de calcul

Coût prohibitif hors grandes organisations

Hors scope

Le fine-tuning est le geste standard en entreprise. On adapte — on n'invente pas.

Ce que le fine-tuning change concrètement

Modèle de base

Llama 3.2 1B
Généraliste

+

Dataset métier

100 exemples FastIA
instruction → JSON

→

Modèle fine-tuné

Spécialisé FastIA
sortie JSON garantie

AVANT FINE-TUNING

"Notre serveur est tombé."

→ Réponse narrative générique, format imprévisible, longueur variable

APRÈS FINE-TUNING

"Notre serveur est tombé."

→ {"categorie": "Support technique", "priorite": "haute",
"reponse_suggeree": "..."}
}

Le problème du fine-tuning complet

Llama 3.2 1B contient **1 milliard de paramètres**.
Un fine-tuning complet modifie tous ces paramètres.

CONSÉQUENCES

Mémoire GPU : 16 à 32 Go minimum
Temps d'entraînement : plusieurs heures ou jours
Risque d'oublier les connaissances générales
Stockage : copie complète du modèle par version

Est-il vraiment nécessaire de modifier 1 milliard de paramètres pour apprendre à classer 5 catégories de demandes ?

RÉPONSE DE LORA (2021)

Non. Les adaptations nécessaires à une tâche spécifique peuvent être représentées par des matrices de très faible rang.
On n'a pas besoin de tout modifier.

LoRA

Comment fonctionne LoRA

On ajoute de petites matrices d'adaptation sur les couches d'attention — sans toucher aux poids originaux.

Poids originaux W

Gelés — non modifiés

+

Matrice A × Matrice B

Entraînables — très petites

=

W + AB

Comportement adapté

R — RANG

Taille des matrices A et B. On utilise $r=8$. Plus r est grand, plus le modèle peut apprendre — et plus il consomme de mémoire.

LORA_ALPHA

Facteur d'échelle appliqué aux matrices. Généralement $2 \times r$. Contrôle l'intensité de l'adaptation appliquée.

TARGET_MODULES

Couches sur lesquelles LoRA est appliqué. On cible `q_proj` et `v_proj` — les projections de l'attention.

LoRA en chiffres

Approche	Paramètres entraînés
Fine-tuning complet	1 000 000 000
LoRA r=8	~8 000 000
Ratio	~0.8%

On entraîne moins de 1% des paramètres pour obtenir un modèle spécialisé. Le reste reste intact.

AVANTAGES

Fonctionne sur hardware modeste
Rapide — quelques dizaines de minutes
Le modèle de base reste inchangé
Plusieurs adaptateurs pour un même modèle

DANS NOS BRIEFS

Llama 3.2 1B + LoRA r=8
Fonctionne sur votre machine
Entraînement en moins d'une heure
Adaptateur sauvegardé : ~20 Mo

HuggingFace Hub — le registre des modèles

CE QU'ON Y TROUVE

Plus de 500 000 modèles publics
Llama, Mistral, BERT, Whisper, CLIP...
Datasets et espaces de démonstration
Documentation et exemples de code

POUR NOS BRIEFS

meta-llama/Llama-3.2-1B
Accès via token HuggingFace
Accepter les conditions d'utilisation Meta
huggingface.co → Settings → Access Tokens

HuggingFace Hub

↓ `AutoTokenizer.from_pretrained()`

Tokenizer
texte → tokens



`AutoModelForCausalLM.from_pretrained()`

Modèle de base
poids pré-entraînés

↓ `get_peft_model()`

Modèle + adaptateur LoRA
prêt à entraîner

HuggingFace Hub

La bibliothèque PEFT

PEFT (Parameter-Efficient Fine-Tuning) implémente LoRA et d'autres techniques d'adaptation efficace.

CE QU'ELLE FAIT

- Ajoute les matrices LoRA sur le modèle de base
- Gèle automatiquement les poids originaux
- N'expose que les paramètres entraînables au Trainer
- Sauvegarde uniquement l'adaptateur (~20 Mo)

DANS LE NOTEBOOK

- `LoraConfig` — paramètres `r`, `alpha`, `dropout`
- `get_peft_model()` — applique LoRA
- `print_trainable_parameters()` — vérifie le ratio
- `model.save_pretrained()` — sauvegarde l'adaptateur

A la sauvegarde, seuls les poids LoRA sont écrits sur disque. Pour utiliser le modèle, on charge le modèle de base et on applique l'adaptateur par-dessus.

Le problème à résoudre

FastIA reçoit chaque jour des centaines de demandes clients en texte libre. Elles sont traitées manuellement : lecture, catégorisation, routage vers le bon interlocuteur, rédaction d'une première réponse. C'est lent et source d'erreurs.

OBJECTIF

Fine-tuner Llama 3.2 1B pour qu'il analyse chaque demande entrante et génère automatiquement une réponse structurée.

SORTIE ATTENDUE

```
{  
  "categorie": "Support technique",  
  "priorite": "haute",  
  "reponse_suggeree": "..."  
}
```

CATÉGORIE 1

Support technique

CATÉGORIE 2

Demande commerciale

CATÉGORIE 3

Demande de transformation

CATÉGORIE 4

Réclamation

CATÉGORIE 5

Information générale

Le dataset fourni

STRUCTURE

100 exemples au format JSON
20 exemples par catégorie
Priorité haute ou normale
Demandes en français, styles variés

FORMAT DE CHAQUE EXEMPLE

```
{  
  "input": "texte de la demande",  
  "output": {  
    "categorie": "...",  
    "priorite": "...",  
    "reponse_suggeree": "..."  
  }  
}
```

Ce dataset est volontairement petit. L'objectif est de comprendre ce qu'on peut faire avec peu de données — et d'identifier ce qui ne fonctionne pas. C'est le travail du Brief 1.

PROMPT TEMPLATE — FORMAT LLAMA

```
<s>[INST] Tu es un assistant FastIA.  
Analyse la demande suivante et réponds  
uniquement en JSON valide avec les champs :  
categorie, priorite, reponse_suggeree.
```

```
Demande : {input} [/INST]  
{output_json} </s>
```

Cas FastIA

Les trois briefs du module

BRIEF 1 — DÉMARRE AUJOURD'HUI

Exploration du dataset
Construction du prompt template
Fine-tuning initial + MLflow
Evaluation quantitative et qualitative
Diagnostic et actions correctives

BRIEF 2 — DISTANCIEL

Mise en oeuvre des actions correctives
Compléter le dataset et/ou ajuster les
hyperparamètres
Réentraînement — Run 2
Comparaison MLflow Run 1 vs Run 2
Exposition via FastAPI

BRIEF 3 — DISTANCIEL

Logging Loguru
Tests fonctionnels Pytest
Conteneurisation Docker
Documentation complète
README + GitHub

Le Brief 1 commence maintenant. Ouvrez `notebook_brief1_finetuning.ipynb` et commencez par la section 2 — Exploration du dataset.

Ce que vous savez faire ce soir

- Expliquer pourquoi l'architecture Transformer a remplacé les RNN
- Décrire le mécanisme d'attention Q/K/V sans les formules
- Distinguer encoder et decoder et leurs cas d'usage respectifs
- Expliquer la différence entre pré-entraînement et fine-tuning
- Expliquer pourquoi LoRA permet de fine-tuner sur hardware modeste
- Charger un modèle depuis HuggingFace Hub et appliquer LoRA avec PEFT
- Lancer un premier fine-tuning tracké dans MLflow

Suite — Distanciel : finaliser le Brief 1 (évaluation + diagnostic), puis attaquer le Brief 2 (actions correctives + FastAPI).